מספר התלמיד הנבחן
רשום את כל תשע הספרות

הדבק כאן את
מדבקת הנבחן

האוניברסיטה
הפתוחה

כ"ב בתמוז תשע"ח

מס' שאלון - 521

5

ביולי 2018

סמסטר 2018ב

מס' מועד 85

20905 / 4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **11** עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

שימו לב:

את תשובותיכם רשמו אך ורק במחברת הבחינה.

כתבו בכתב יד ברור ומסודר.

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (25 נקודות)

הרחיבו את שפת LET (שפת "ויהי") כך שביטוי if יבוצע על פי הדקדוק הבא:

להלן הדקדוק המגדיר את התחביר החדש עבור ביטוי if:

Expression* ::= if [{*Expression* }$^{+(,)}$] then *Expression
else *Expression*

if-exp (bools exp1 exp2)

כאשר,

- bools רשימה של ביטויים בוליאניים.
- exp1 ביטוי שערכו יוחזר כאשר רשימת הביטויים הבוליאניים הכילה **לכל היותר** ביטוי אחד שערכו false.
- exp2 ביטוי שערכו יוחזר אם ברשימת הביטויים הבוליאניים היו לפחות 2 ביטויים שערכם false

להלן דוגמא להמחשת השימוש בביטוי החדש:

```
let x= 7
in
  let z= if [ zero? (-(x,7)) , zero? (0) , zero? (9)] then 10 else 20
   in  -(z,x)
```

בדוגמה זו, ישנם 3 ביטויים בוליאניים, השניים הראשונים ערכם הוא true ורק השלישי הוא false ולכן ערכו של הביטוי הוא 10, ולכן משתנה z יקבל 10. ולכן ערכה של כל הדוגמה הוא (num-val 3)

א. (7 נקודות)

להלן נתונים ה-scanner, וה-parser של שפת "ויהי". כתבו מהו השינוי הדרוש בדקדוק של ביטוי if כך שיתאים להגדרת הדקדוק החדש. **<u>את תשובותיכם כתבו אך ורק במחברת הבחינה.</u>**

ה-scanner:

```
(define the-lexical-spec
   '((whitespace (whitespace) skip)
    (comment ("%" (arbno (not #\newline))) skip)
    (identifier (letter (arbno (or letter digit "_" "-" "?")))  symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
   ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
   (expression (number) const-exp)
   (expression ("-" "(" expression "," expression ")") diff-exp)
   (expression ("zero?" "(" expression ")") zero?-exp)
   (expression ("if" expression "then" expression "else" expression) if-exp)
   (expression (identifier) var-exp)
   (expression ("let" identifier "=" expression "in" expression) let-exp)
  ))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"ויהי"** כפי שמופיעה בספר הלימוד. יש לעדכנה כך שתחשב את ביטוי if על פי הדקדוק החדש. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "ויהי".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

הפרוצדורה *value-of* (כפי שמופיעה בספר בעמוד 71):

```
;;value-of : Exp x Env ----> ExpVal
 (define value-of
  (lambda (exp env)
   (cases expression exp
    (const-exp (num) (num-val num))
    (var-exp (var) (apply-env env var))
    (diff-exp (exp1 exp2)
       (let ((val1 (value-of exp1 env))
             (val2 (value-of exp2 env)))
         (let ((num1 (expval->num val1))
               (num2 (expval->num val2)))
          (num-val (- num1 num2)))))
    (zero?-exp (exp1)
      (let ((val1 (value-of exp1 env)))
       (let ((num1 (expval->num val1)))
         (if (zero? num1) (bool-val #t)  (bool-val #f)))))
    (if-exp (exp1 exp2 exp3)
      (let ((val1 (value-of exp1 env)))
        (if (expval->bool val1) (value-of exp2 env) (value-of exp3 env))))
    (let-exp (var exp1 body)
      (let ((val1 (value-of exp1 env)))
        (value-of body (extend-env var val1 env))))
    )))
```

שאלה 2 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

ברצוננו להרחיב את השפה עם ביטוי חדש שתחבירו נתון על-ידי הדקדוק הבא:

Expression ::= **procsum** ***Expression*** **[{** ***Expression*** **}**$^{*(;)}$**]**

```
procsum-exp (exp prcs)
```

כאשר,

- exp הוא ביטוי שתוצאתו היא מספר.
- prcs הוא אוסף של 0 או יותר ביטויים שכל אחד מהם מתאר פרוצדורה המקבלת מספר ומחזירה מספר.

הביטוי החדש מפעיל את הפרוצדורות prcs (אחת אחרי השנייה) על הביטוי exp. ומחזיר את סכום ערכי ההחזר של הפעלת הפרוצדורות. במקרה שהאוסף exps ריק יוחזר ערכו של exp.

להלן דוגמה להמחשת השימוש בביטוי החדש:

```
let p1= proc (x) –(x,1)
in
  let p2 = proc (y) –(y,6)
  in
    procsum  -(5,1)  [p1; p2]
```

בדוגמה זו, נתון ביטוי שערכו הוא 4 ואוסף של 2 פרוצדורות. ערכו של ביטוי זה יהיה שווה לסכום של תוצאות הפעלת 2 הפרוצדורות על הערך 4. הפרוצדורה הראשונה תחזיר 3, והפרוצדורה השניה תחזיר 2-, ולכן ערכו של כל הביטוי יהיה (num-val 1)

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וישגור**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
      (define the-grammar
          '((program (expression) a-program)

            (expression (number) const-exp)
            (expression
              ("-" "(" expression "," expression ")")
      diff-exp)

            (expression
             ("zero?" "(" expression ")")
      zero?-exp)

            (expression
             ("if" expression "then" expression "else" expression)
      if-exp)

            (expression (identifier) var-exp)

            (expression
             ("let" identifier "=" expression "in" expression)
      let-exp)

            (expression
             ("proc" "(" identifier ")" expression)
      proc-exp)

             (expression
             ("(" expression expression ")")
      call-exp)
```

```
(expression
```

את התשובה יש לכתוב במחברת הבחינה

```
procsum-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"וישגור"** . ולאחריה פרוצדורת העזר apply-procedure כפי שמופיעה בספר הלימוד בעמוד 79. יש להשלים את החסר בפרוצדורה value-of. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וישגור".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;;value-of : Exp x Env→ExpVal
(define value-of
(lambda (exp env)
(cases expression exp
(const-exp (num) (num-val num))
(var-exp (var) (apply-env env var))
(diff-exp (exp1 exp2)
  (let ((val1 (value-of exp1 env))
(val2 (value-of exp2 env)))
(let ((num1 (expval->num val1))
   (num2 (expval->num val2)))
(num-val(- num1 num2)))))
(zero?-exp (exp1)
(let ((val1 (value-of exp1 env)))
(let ((num1 (expval->num val1)))
(if (zero? num1)(bool-val #t)(bool-val #f)))))
(if-exp (exp1 exp2 exp3)
 (let ((val1 (value-of exp1 env)))
 (if (expval->bool val1)
(value-of exp2 env)
(value-of exp3 env))))
       (let-exp (var exp1 body)
 (let ((val1 (value-of exp1 env)))
 (value-of body(extend-env var val1 env))))

     (proc-exp (var body)
          (proc-val (procedure var body env)))

(call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
                (arg (value-of rand env)))
            (apply-procedure proc arg)))
```

```
(procsum-exp   (____________________________________)

                 את התשובה יש לכתוב במחברת הבחינה

                                                    ))))
```

```
;; apply-procedure : Proc * ExpVal ->ExpVal
;; Page: 79
  (define apply-procedure
    (lambda (proc1 val)
      (cases proc proc1
        (procedure (var body saved-env)
          (value-of body (extend-env var val saved-env))))))
```

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "**וירמוז**"(IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם range.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression ::= range (identifier [{ *Expression*}$^{+(,)}$]) *Expression*

range-exp (tmpid exps body)

לביטוי **range-exp** המרכיבים הבאים:

- **tmpid** – משתנה זמני שמוכר רק במסגרת פעולת ביטוי range-exp.
- **exps** – רשימת ביטויים (לפחות אחד) המופרדים בפסיק.
- **body** – ביטוי אותו יש לבצע שוב ושוב

שלבי חישוב ביטוי range-exp הם:

המשתנה הזמני יקבל כערך את הערכים ברשימה exps, בכל פעם ערך יחיד (החל מהראשון ועד לאחרון) , עבור כל ערך שמקבל המשתנה הזמני, נדרש לבצע את הביטוי המתואר ע"י body.

הערך המוחזר מכל הביטוי הוא ערכו האחרון של body.

להלן דוגמא :

```
let z= 10
in let y= range (x [1,5,-2,-(3,8)]) set z= -(z,x)
   in  y
```

בדוגמה זו, מוגדר משתנה z שערכו 10.

לאחר מכן, מוגדר משתנה y המקבל את ערכו של ביטוי range.

ביטוי ה-range מגדיר משתנה זמני x, בתחילה x מקבל ערך של 1 ועם ערך זה מבוצע ה-body שבמקרה זה משנה את ערכו של z להיות 9 (10-1) . לאחר מכן x משתנה ל- 5, ואז שוב מבוצע ה-body ששוב את ישנה את z ל- 4 (9-5). לאחר מכן, ערכו של x הוא 2- וערכו של z יהיה 6 (4+2). ולבסוף ערכו של x יהיה 5- וערכו של z 11 (6+5). ולכן y יקבל 11. ולכן ערכה של כל הדוגמה הוא: (num-val 11)

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וירמוז**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
'((program (expression) a-program)
  (expression (number) const-exp)
  (expression ("-" "(" expression "," expression ")")
    diff-exp)
  (expression ("zero?" "(" expression ")") zero?-exp)
  (expression
   ("if" expression "then" expression "else" expression)
     if-exp)
  (expression (identifier) var-exp)
  (expression ("let" identifier "=" expression "in" expression)
     let-exp)
  (expression ("proc" "(" identifier ")" expression)
     proc-exp)
  (expression ("(" expression expression ")") call-exp)
  (expression
        ("letrec"
          (arbno identifier "(" identifier ")" "=" expression)
           "in" expression)
letrec-exp)
(expression ("set" identifier "=" expression)
    assign-exp)
(expression (
```

את התשובה כיתבו במחברת הבחינה

```
)
  range-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו- *apply-procedure* של שפת **"וירמוז".**

הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.

<u>את התשובה כיתבו במחברת הבחינה</u>.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
  (define value-of
    (lambda (exp env)
      (cases expression exp
        (const-exp (num) (num-val num))

       (var-exp (var) (deref (apply-env env var)))

        (diff-exp (exp1 exp2)
          (let ((val1 (value-of exp1 env))
                (val2 (value-of exp2 env)))
            (let ((num1 (expval->num val1))
                  (num2 (expval->num val2)))
              (num-val
                (- num1 num2)))))

        (zero?-exp (exp1)
          (let ((val1 (value-of exp1 env)))
            (let ((num1 (expval->num val1)))
              (if (zero? num1)
                (bool-val #t)
                (bool-val #f)))))

        (if-exp (exp1 exp2 exp3)
          (let ((val1 (value-of exp1 env)))
            (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

        (let-exp (var exp1 body)
          (let ((v1 (value-of exp1 env)))
            (value-of body
              (extend-env var (newref v1) env))))

        (proc-exp (var body)
          (proc-val (procedure var body env)))

        (call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
                (arg (value-of rand env)))
            (apply-procedure proc arg)))
```

```
      (letrec-exp (p-names b-vars p-bodies letrec-body)
          (value-of letrec-body
            (extend-env-rec* p-names b-vars p-bodies env)))

        (assign-exp (var exp1)
          (begin
            (setref!
              (apply-env env var)
              (value-of exp1 env))
            (num-val 27))))))


(define apply-procedure
      (lambda (proc1 val)
         (cases proc proc1
            (procedure (var body saved-env)
               (value-of body
                  (extend-env var (newref val) saved-env))))))
```

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let p = proc (w:?) (w 7)
in
 (p proc (x:?) -(x,7))
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !